Проект 00 — TensorCore: UML Data Flow

microgradd/engine.py (MIT) · tinygrad/tensor.py (MIT) · nanoGPT/train.py (MIT)

Диаграмма 1 — Архитектура классов: три слоя TensorCore

TensorStorage = плоский float[]. TensorShape = как читать массив. Tensor = данные+форма+autograd.

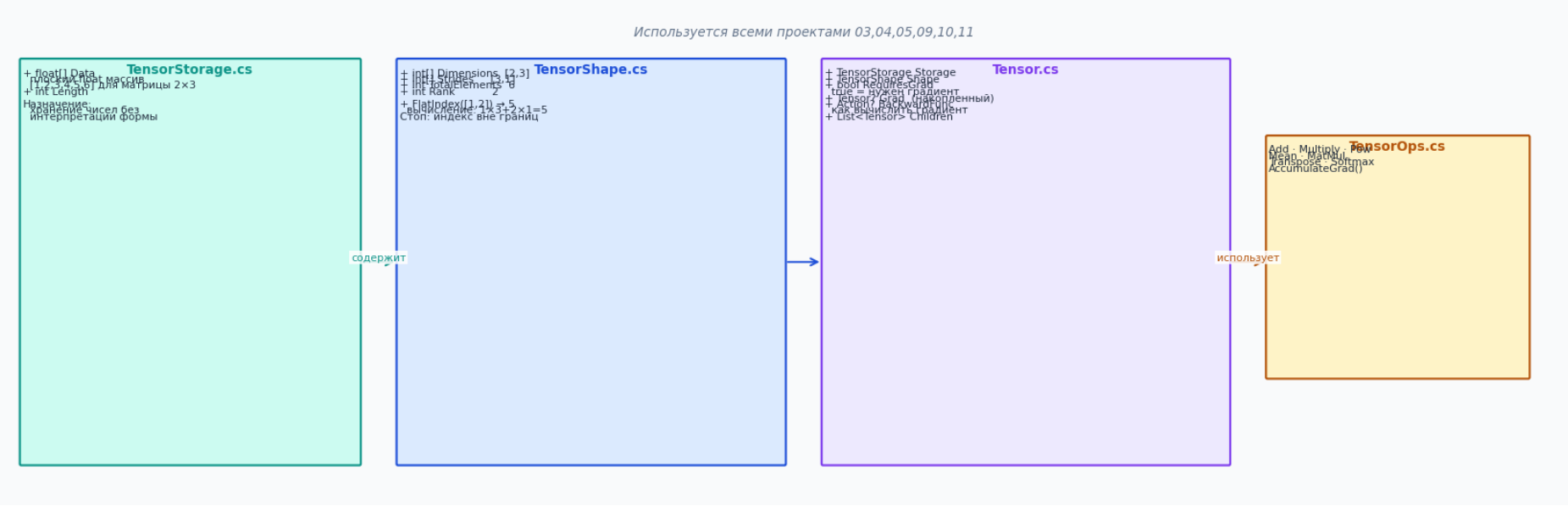


Диаграмма 2 — Forward Pass: последовательность вызовов через Linear слой

Каждая операция: (1) вычисляет результат (2) регистрирует BackwardFunc в Children. Источник: nanoGPT/model.py

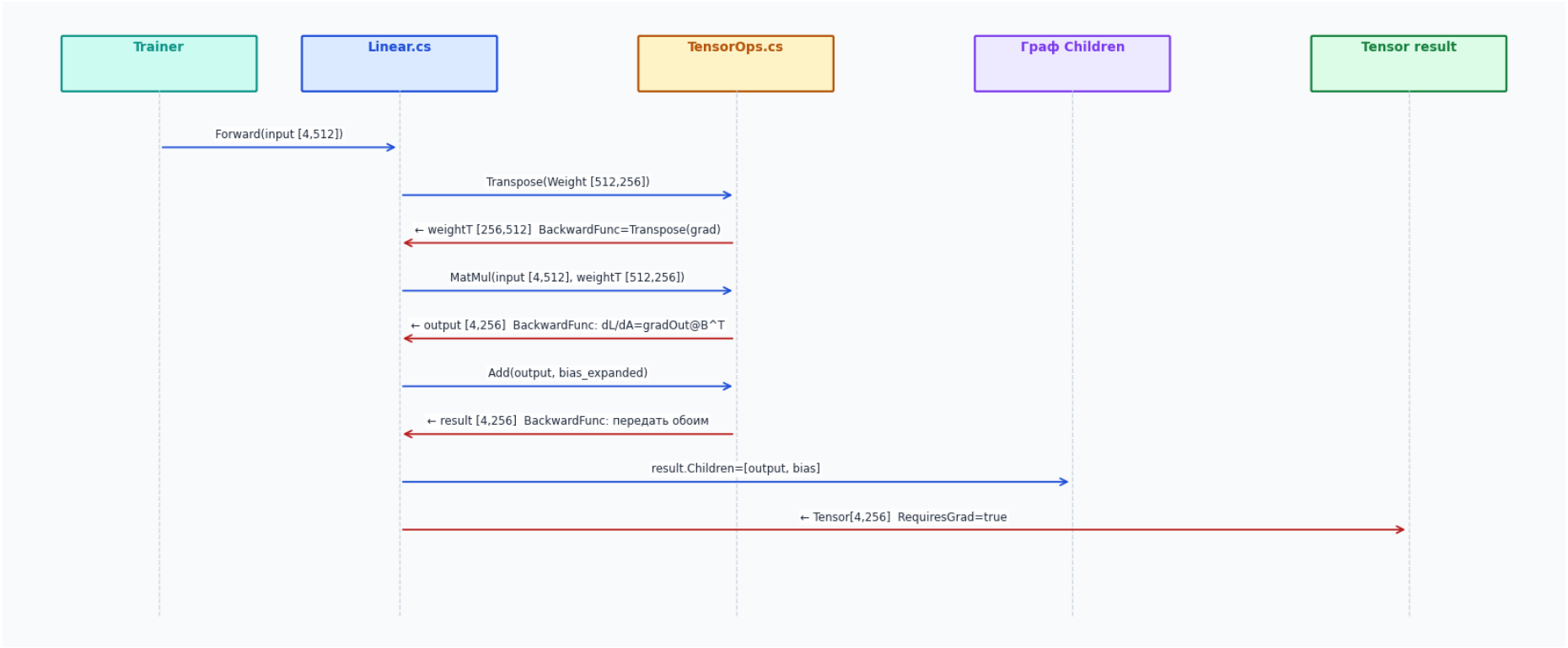
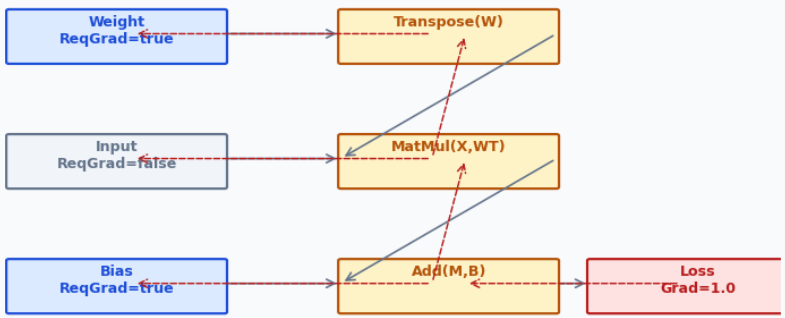

Trainer
Linear.cs
TensorOps.cs
Граф Children
Tensor result
Forward(input [4,512])
Transpose(Weight [512,256])
← weightT [256,512] BackwardFunc=Transpose(grad)
MatMul(input [4,512], weightT [512,256])
← output [4,256] BackwardFunc: dL/dA=gradOut@B^T
Add(output, bias_expanded)
← result [4,256] BackwardFunc: передать обоим
result.Children=[output, bias]
← Tensor[4,256] RequiresGrad=true

Диаграмма 3 — Backward Pass: граф вычислений и правила градиентов

Chain rule (правило цепочки): от выхода к входу. TopologicalSort → BackwardFunc каждого узла. Источник: micrograd/engine.py

Граф вычислений



Правила градиентов (backward functions)

Loss.backward()

Grad = tensor([1.0]) ← начальный градиент

Add.BackwardFunc:

grad_M = grad_out

grad_B += sum(grad_out, axis=0)

MatMul.BackwardFunc:

grad_X = grad_out @ W

grad_WT = X^T @ grad_out

Transpose.BackwardFunc:

grad_W = Transpose(grad_WT)

AccumulateGrad(W,g):

W.Grad += g (накапливаем, т.к. несколько путей)

AdamW.Step():

W -= lr × m̂/(√v̂ + ε) ← обновление весов